

Capítulo 1

Preliminares

En este capítulo se presentan las definiciones y conceptos que serán usados en el resto del trabajo. (Primero¹) se introducen las nociones básicas de la teoría de lenguajes: alfabetos, cadenas, lenguajes, gramáticas y la jerarquía de Chomsky. (Después²) se presentan los autómatas finitos y se enuncia el lema del bombeo para lenguajes libres del contexto, que se emplea en los capítulos posteriores para situar al lenguaje de las fórmulas booleanas satisfacibles dentro de la jerarquía. (Por último³) COMA se introduce la complejidad computacional y se presenta el problema de la satisfacibilidad booleana con sus variantes polinomiales.

1: Pon referencia a la sección con el comando ref.

2: Pon referencia a la sección con el comando ref.

3: Pon referencia a la sección con el comando ref.

1.1. Teoría de Lenguajes

En esta sección se presentan los conceptos de la teoría de lenguajes que sirven de base al contenido de los capítulos posteriores. La sección se organiza en tres partes. Primero se introducen los conceptos de alfabeto, cadena y lenguaje, junto con los problemas de la palabra y del vacío. Después se definen las gramáticas, un mecanismo que permite describir lenguajes formales. Para finalizar se presenta la jerarquía de Chomsky, que clasifica los lenguajes formales según (su poder de generación^{Q1}).

Q1: ¿Qué es esto?

1.1.1. Conceptos básicos

Los conceptos básicos de la teoría de lenguajes formales son: alfabeto, cadena y lenguaje. Un **alfabeto**, denotado como Σ , es un conjunto finito y no vacío de símbolos. Una **cadena** es una sucesión finita de símbolos del alfabeto. Un **lenguaje** es un conjunto de cadenas definido sobre un alfabeto. Por ejemplo, el alfabeto $\Sigma = \{0, 1\}$ está formado por los símbolos 0 y 1; 11 y 101 son cadenas sobre el alfabeto Σ , y un ejemplo de lenguaje es el conjunto de cadenas formadas por (de) 0 y 1 que terminan en 1.

El símbolo ε representa la **cadena vacía**. La **longitud** de una cadena w , (denotada⁴) $|w|$, es la cantidad de símbolos que la componen; en particular, $|\varepsilon| = 0$. NUEVO PÁRRAFO.

Dadas dos cadenas w_1 y w_2 , la **concatenación** w_1w_2 se obtiene al escribir los símbolos de w_1 seguidos de los símbolos de w_2 . Dada una cadena w y un entero $n \geq 0$, por convenio w^n representa la cadena resultante de concatenar w un total de n veces, con $w^0 = \varepsilon$. NUEVO PÁRRAFO.

4: Voz pasiva

Si Σ es un alfabeto, Σ^+ denota el conjunto de todas las cadenas de uno o más símbolos que se pueden formar sobre Σ , y Σ^* denota la unión de Σ^+ y $\{\varepsilon\}$.

Dado que los lenguajes son conjuntos, las operaciones sobre conjuntos también se definen para lenguajes: unión, intersección y complemento [6]. A continuación se formalizan (dos problemas asociados a un lenguaje^{Q2}) que se utilizan de manera recurrente en los capítulos posteriores: determinar si una cadena pertenece al lenguaje y determinar si el lenguaje contiene algún elemento.

Q2: ¿Qué es esto?

Definición 1.1. El **problema de la palabra** consiste en determinar si una cadena pertenece a un lenguaje dado.

Por ejemplo, dado el lenguaje L de las cadenas binarias que terminan en 0, el problema de la palabra para la cadena 1100100 consiste en determinar si $1100100 \in L$.

Definición 1.2. El **problema del vacío** consiste en determinar si un lenguaje es vacío.

Por ejemplo, el problema del vacío para el lenguaje formado por (conjunto de) los números pares mayores que 5 que además son primos COMA consiste en decidir si tal lenguaje (conjunto) tiene algún elemento. Los antecedentes del presente trabajo, que se revisan en el capítulo ??, emplean el problema del vacío (sobre el conjunto de interpretaciones^{Q3}) que (hacen verdadera^{Q4}) a (una fórmula booleana^{Q5}) para decidir si la fórmula es satisfacible.

Q3: ¿Qué es esto?

Q4: ¿Qué es esto?

Q5: ¿Qué es esto?

Q6: ¿Qué es esto?

Una vez introducidos los conceptos básicos y los dos problemas anteriores, a continuación se presentan las gramáticas, un mecanismo que permite describir los elementos de un lenguaje mediante (reglas de reescritura^{Q6}) .

1.1.2. Gramáticas

Los lenguajes formales se pueden describir mediante **gramáticas**, un mecanismo que permite generar los elementos de un lenguaje a partir de un símbolo inicial y un conjunto de (reglas de reescritura^{Q7}) .

Q7: ¿Qué es esto?

Definición 1.3. Una **gramática** es un formalismo utilizado para describir lenguajes formales. Se define como una 4-tupla

$$G = (N, \Sigma, P, S),$$

donde

- N es un conjunto finito de **símbolos llamados símbolos no terminales**;
- Σ es un conjunto finito de **símbolos llamados símbolos terminales**, que constituyen el alfabeto sobre el que se construyen las cadenas del lenguaje (PUNTO. Se debe cumplir (y se cumple) que $N \cap \Sigma = \emptyset$;
- P es un conjunto finito de **reglas de producción** de la forma

$$\alpha \rightarrow \beta, \quad \text{con } \alpha, \beta \in (N \cup \Sigma)^*;$$

- $S \in N$ es el **símbolo inicial**, que define (el punto de partida^{Q8}) para (derivar cadenas del lenguaje^{Q9}) .

Q8: ¿Qué es esto?

Q9: ¿Qué es esto?

Una **derivación** en la gramática consiste en seleccionar una regla de producción $\alpha \rightarrow \beta$ y sustituir una ocurrencia de α en una cadena $w \in (\Sigma \cup N)^+$ por β [6].

AQUÍ UN EJEMPLITO VENDRÍA BIEN.MS

Se dice que una cadena $w \in \Sigma^*$ se genera por la gramática G si existe (una secuencia de derivaciones^{Q10}) que comienza con S y termina con la cadena w . El lenguaje generado por una gramática G se denota

$$L(G) = \{w \in \Sigma^* \mid S \xrightarrow{+} w\},$$

donde $\xrightarrow{+}$ indica una (derivación de uno o más pasos^{Q11}).

(Un formalismo gramatical^{Q12}) también puede definir, para cada cadena de entrada, si existe una secuencia de derivaciones que permite obtenerla [REF]. En tal caso se dice que la cadena (es reconocida⁵) por el formalismo, y el conjunto de todas las cadenas reconocidas conforma el **lenguaje reconocido**. (La distinción entre generación y reconocimiento se emplea en los capítulos posteriores, pues las gramáticas de concatenación de rango se formulan directamente como un mecanismo de reconocimiento.⁶)

Una vez presentadas las gramáticas, a continuación se expone la jerarquía de Chomsky, que clasifica a los lenguajes formales según el poder de generación de las gramáticas que los describen.

1.1.3. Jerarquía de Chomsky

La **Jerarquía de Chomsky** [7] clasifica los lenguajes en cuatro tipos, según las restricciones en las reglas de producción de las gramáticas que los generan.

El primer tipo son las **gramáticas irrestrictas**, que no imponen restricciones sobre sus reglas de producción. Cada regla tiene la forma $\alpha \rightarrow \beta$, donde $\alpha, \beta \in (N \cup \Sigma)^*$ y $\alpha \neq \varepsilon$. Todo lenguaje generado por una gramática irrestricta se denomina **lenguaje recursivamente enumerable** [REF].

El segundo tipo son las **gramáticas dependientes del contexto**. En estas, cada regla tiene la forma $\alpha A \gamma \rightarrow \alpha \beta \gamma$, donde $A \in N$, $\alpha, \beta, \gamma \in (N \cup \Sigma)^*$, y $|\beta| \geq 1$. Todo lenguaje generado por una gramática dependiente del contexto se denomina **lenguaje dependiente del contexto**. Todo lenguaje dependiente del contexto es también un lenguaje recursivamente enumerable. (En el capítulo ?? se muestra que el lenguaje de las fórmulas booleanas satisfacibles no es libre del contexto, por lo que necesariamente se sitúa en alguno de estos dos tipos superiores.⁷)

El tercer tipo son las **gramáticas libres del contexto** (CFG). En estas, cada regla tiene la forma $A \rightarrow \beta$, donde $A \in N$ y $\beta \in (N \cup \Sigma)^*$. Todo lenguaje generado por una gramática libre del contexto se denomina **lenguaje libre del contexto**. Todo lenguaje libre del contexto es también un lenguaje dependiente del contexto [REF].**NUEVO PÁRRAFO.**

Las CFG son **el punto de referencia obligado** para los formalismos que se estudian en el capítulo ??, ya que las gramáticas de concatenación de rango se introducen como una extensión de las CFG. Además, en [4] se emplea una CFG para resolver instancias del problema de la satisfacibilidad booleana.

El cuarto tipo son las **gramáticas regulares**. En estas, las reglas de producción tienen la forma

$$A \rightarrow aB \quad \text{o} \quad A \rightarrow a,$$

Q10: ¿Qué es esto?

Q11: ¿Qué es esto?

Q12: ¿Qué es esto?

5: Voz pasiva

6: Creo que esto aquí hace más ruido que los problemas que resuelve... quizás no haga mucha falta... pero si lo quitas, a lo mejor te pregunto para qué lo estás poniendo... ¡monito con los ojos tapados!

7: ¿tú lo demuestras otra vez?

donde $A, B \in N$ y $a \in \Sigma$. **NUEVO PÁRRAFO.**

Todo lenguaje generado por una gramática regular se denomina **lenguaje regular**. Todo lenguaje regular es un lenguaje libre del contexto. Los lenguajes regulares se relacionan con los autómatas finitos, que se presentan en la sección 1.2.

(Un lenguaje A es más expresivo que un lenguaje B ⁸) si todas las cadenas que pertenecen a B también pertenecen a A y existen cadenas que pertenecen a A y no a B . Por ejemplo, (los lenguajes libres del contexto⁹) son más expresivos que los lenguajes regulares [6].

La diferencia entre las familias de los lenguajes de la jerarquía de Chomsky se puede ilustrar con una familia de lenguajes derivada del lenguaje Copy. Dado un alfabeto Σ y un entero $k \geq 1$, se define el lenguaje

$$L_{\text{copy}}^k = \{w^k \mid w \in \Sigma^*\}.$$

El caso $k = 1$ corresponde a $L_{\text{copy}}^1 = \Sigma^*$, que es un lenguaje regular. Para $k \geq 2$, el lenguaje L_{copy}^k no es libre del contexto, aunque sí es dependiente del contexto [6]. En el capítulo ?? se emplea este lenguaje para ilustrar el poder expresivo de las gramáticas de concatenación de rango.

La relación de inclusión entre los lenguajes de la jerarquía se ilustra en la figura 1.1 (, donde las elipses que aparecen dentro de otras significan que la de adentro está contenida en la de afuera¹⁰).

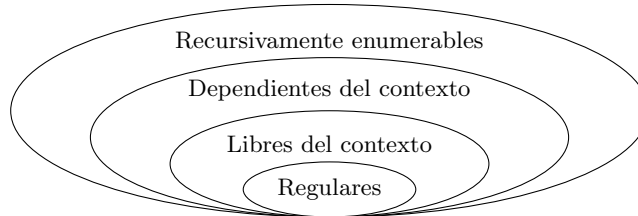


Figura 1.1: Jerarquía de Chomsky.

Cada clase de la jerarquía de Chomsky admite un (~~modelo de máquina abstracta~~) que reconoce sus lenguajes. En la próxima sección se presenta el modelo asociado a los lenguajes regulares y (se enuncia una herramienta estándar¹¹) para probar que un lenguaje no es libre del contexto.

1.2. Autómatas finitos y propiedades de los lenguajes libres del contexto

Un **autómata** es una (~~máquina abstracta~~^{Q13}) que procesa cadenas de símbolos de un alfabeto ((finito)¹²) y determina si una cadena pertenece a un lenguaje [6]. **NUEVO PÁRRAFO.**

Cada clase de la jerarquía de Chomsky tiene un modelo de autómata equivalente: los lenguajes recursivamente enumerables se reconocen por una máquina de Turing [6], los lenguajes dependientes del contexto por una máquina de Turing linealmente acotada ([6]¹³), los lenguajes libres del contexto por un autómata de pila [6] y los lenguajes regulares por

8: ¿de dónde sacaste esta definición? Creo que lo que son más expresivos son los formalismos no los lenguajes.

9: Ahhhhh... sí, sí... estás hablando de familias de lenguajes o formalismos gramaticales. Eso creo que se debería definir en algún lugar, o si no definir, al menos decir que le vamos a llamar de esa forma.

10: No lo escribas así, que está en candela, pero la idea debería ir por ahí.

11: No seas misterioso. Cuando dices que más adelante vas a usar algo, deberías decir qué es ese algo. La frase es: el misterio y la incertidumbre son buenas para las novelas, pero no para los documentos científicos :D. Así que aquí menciono explícitamente que es el lema del bombeo para lenguajes libres del contexto.

Q13: ¿Qué es esto?

12: ya habías dicho que todos los alfabetos son finitos.

13: Como las 4 referencias son la misma, la puedes poner una única vez justo después de autómata equivalente.

un autómata finito [6]. En esta sección se define el autómata finito y se enuncia el lema del bombeo para lenguajes libres del contexto, que se emplea en los capítulos posteriores.

Definición 1.4. Un **autómata finito** [6] es un modelo matemático (que permite reconocer si una cadena pertenece a un lenguaje regular¹⁴). Se define como una 5-tupla

$$\mathcal{A} = (Q, \Sigma, \delta, q_0, F),$$

donde Q es un conjunto finito de **estados**, Σ es un **(el) alfabeto (finito) llamado alfabeto de entrada**, $\delta : Q \times \Sigma \rightarrow Q$ es una **función de transición** que define (cómo el autómata cambia de estado en función del símbolo leído^{Q14}), $q_0 \in Q$ es el **estado inicial** (desde donde comienza la computación^{Q15}) y $F \subseteq Q$ es el conjunto de **estados de aceptación**.

El autómata (comienza¹⁵) en el estado inicial q_0 (y lee los símbolos de la cadena uno a uno^{Q16}). En cada paso, la función de transición δ determina, a partir (del símbolo actual^{Q17}) y (del estado actual^{Q18}), (el estado al que pasa el autómata^{Q19}). (Al finalizar la lectura^{Q20}) (del último símbolo^{Q21}), (si el autómata se encuentra en un estado de F ^{Q22}), la cadena se acepta; en caso contrario, se rechaza.

[Quizás la forma de quitarnos de arriba la pila de ¿que es eso? esos es hacer como hacen el hopcroft y definir configuración y después transición, a partir de la definición de transición. Ahora mismo no tiene mucho valor porque solo has estado hablando de cosas ahí del autómata.]

El resultado que se enuncia a continuación es una herramienta estándar para demostrar que un lenguaje no es libre del contexto. Se emplea en los capítulos posteriores para ubicar (al lenguaje de las fórmulas booleanas satisfacibles^{Q23}) dentro de la jerarquía de Chomsky. [¿Y por qué quisieras hacer eso?]

Teorema 1.1 (Lema del bombeo para lenguajes libres del contexto). Si L es un lenguaje libre del contexto, entonces existe una constante n tal que para toda cadena $t \in L$ con $|t| \geq n$, t puede escribirse como $t = uvwxy$ con $|vwx| \leq n$, $vx \neq \varepsilon$ y $uv^iwx^iy \in L$ para todo $i \geq 0$.

La demostración del teorema anterior se realiza en [6].

Una vez presentadas estas herramientas, a continuación se introducen los conceptos de complejidad computacional que permiten clasificar la dificultad de los problemas asociados (a los lenguajes formales¹⁶). Esta clasificación se emplea para analizar la complejidad del problema de la palabra en los formalismos que se estudian en los capítulos posteriores.

1.3. Complejidad computacional

(En esta sección se definen los conceptos de complejidad computacional que se utilizan en los capítulos posteriores para analizar la complejidad del problema de la palabra. La sección se organiza en tres partes. Primero se presenta la notación asintótica. Después se definen las clases de problemas P , NP y NP -Completo. Por último se define la reducción polinomial y se precisa el problema de reconocimiento.¹⁷)

1.3.1. Notación asintótica

La notación asintótica describe el comportamiento de una función $f(n)$ a medida que n crece hacia el infinito.

14: En realidad esto es un resultado que se deriva de la definición. Si revisas [6] verás que eso es un teorema, así que no debería estar aquí en la definición. Creo que aquí puedes decir directamente que un autómata es una tupla...

Q14: ¿Qué es esto?

Q15: ¿Qué es esto?

15: ¿comienza qué?

Q16: ¿Qué es esto?

Q17: ¿Qué es esto?

Q18: ¿Qué es esto?

Q19: ¿Qué es esto?

Q20: ¿Qué es esto?

Q21: ¿Qué es esto?

Q22: ¿Qué es esto?

Q23: ¿Qué es esto?

16: dicho así parece que es solo para estos problemas. Creo que lo que vas a presentar permite clasificar la dificultad de cualquier tipo de problema... ¿no?

17: Varias veces ha pasado que repites la misma idea en el párrafo del final de la sección anterior y la del inicio de la actual. Debes tener cosas en los dos lugares, pero no exactamente las mismas cosas. Quizás puedes una parte antes y la otra después.

Definición 1.5. Una función $g(n)$ pertenece a $O(f(n))$ si existen constantes positivas c y n_0 tales que

$$g(n) \leq c \cdot f(n) \quad \text{para todo } n \geq n_0.$$

Esta notación proporciona un **(límite superior asintótico^{Q24})** para $g(n)$. Se dice que un algoritmo tiene **tiempo polinomial** si **(puede ejecutarse en complejidad $O(n^k)$ ¹⁸)**, donde n es el tamaño de la entrada del algoritmo y k es una constante. La **complejidad computacional** de un problema se define como la complejidad del algoritmo más eficiente que lo resuelve en el peor caso [REF].

TRANSICIÓN A LO QUE SIGUE.

1.3.2. Clases de problemas

Los problemas computacionales [6] se agrupan en clases según los recursos necesarios para resolverlos. En este trabajo se emplean las clases P , NP , NP -Completo y NP -Duro.

Definición 1.6. Un problema pertenece a la clase P si se puede resolver en tiempo polinomial [6].

Pon ejemplitos.

Definición 1.7. Un problema pertenece a la clase NP si **(su solución se puede verificar^{Q25})** en tiempo polinomial [6].

Pon ejemplitos.

Definición 1.8. Un problema pertenece a la clase NP -Completo si pertenece a NP y, además, cualquier problema en NP **(se puede reducir^{Q26})** a él en tiempo polinomial [6].

Pon ejemplitos.

Definición 1.9. Un problema pertenece a la clase NP -Duro si cualquier problema en NP se puede reducir a él en tiempo polinomial, sin requerirse **(que el problema¹⁹)** pertenezca a NP [6].

Pon ejemplitos.

La relación entre las clases P y NP es uno de los problemas abiertos de la teoría de la computación [6]. **(A la fecha²⁰)** se desconoce si $P = NP$ o si $P \neq NP$. El conjunto de problemas NP -Completo proporciona una base para el estudio de esta pregunta: si algún problema de la clase NP -Completo se puede resolver en tiempo polinomial, entonces todos los problemas de NP también. Los problemas de la clase NP -Duro pueden resultar incluso más difíciles, ya que no se requiere que pertenezcan a NP [REF].

Una vez definidas las clases de problemas, a continuación se define **(la reducción polinomial²¹)** y **(se precisa el problema de reconocimiento²²)**, que será **(el vínculo entre los problemas computacionales y los formalismos gramaticales en los capítulos posteriores^{Q27})**.

Q24: ¿Qué es esto?

18: ¿qué significa que un algoritmo pueda ejecutarse en una complejidad?

Q25: ¿Qué es esto?

Q26: ¿Qué es esto?

19: ¿cuál de ellos, el que está siendo reducido o el que estás analizando? Eso quedó ambiguo.

20: ¿a qué fecha? Fíjate que si resuelves P vs NP , una pila de gente va a querer leer tu tesis, y dentro de 20 años no se va a saber qué significa *A la fecha*.

21: Bueno... aquí tenemos un problema porque eso ya lo usaste por allá arriba... ¿cómo que ahora es que vas a definirlo?

22: ¿qué significa esto? (Aquí sí que no sé qué quisiste decir)

Q27: ¿Qué es esto?

1.3.3. Reducción polinomial y problema de reconocimiento

Una **reducción polinomial** de un problema A a un problema B es (una función computable^{Q28}) en tiempo polinomial que transforma (instancias de A ^{Q29}) en instancias de B , (de modo que la respuesta para la instancia transformada coincide con la respuesta para la instancia original^{Q30}). Las reducciones polinomiales permiten (transferir resultados de complejidad entre problemas^{Q31}) y son la base para definir la clase NP -Completo. Si un problema A se reduce polinomialmente a un problema B y B pertenece a P , entonces A también pertenece a P [REF].

La complejidad del **problema de la palabra**, también llamado (problema de reconocimiento²³), para un formalismo gramatical se define como la complejidad del algoritmo más eficiente que determina, dada una cadena w , si w pertenece al lenguaje descrito por el formalismo.

Todo problema en Ciencia de la Computación se puede reducir a un problema de la palabra, ya que cualquier problema se puede codificar como un lenguaje formal [6]. NUEVO PÁRRAFO.

Por ejemplo, el problema de decidir si un grafo no dirigido G posee un camino hamiltoniano se puede codificar como un lenguaje formal sobre un alfabeto finito. Para ello se fija una representación textual estándar de los grafos, como las listas de adyacencia (sobre un alfabeto de símbolos²⁴), y se define el lenguaje L_{HAM} como el conjunto de todas las cadenas que codifican un grafo con un camino hamiltoniano. Decidir si una cadena dada pertenece a L_{HAM} equivale a decidir si el grafo codificado posee un camino hamiltoniano. NUEVO PÁRRAFO.

(Este planteamiento²⁵) se utiliza en los capítulos posteriores: (las fórmulas booleanas^{Q32}) se codifican como cadenas de un lenguaje formal, por(eon) lo que determinar(decidir) si una fórmula es satisfacible se reduce a determinar(decidir) si (la cadena asociada²⁶) pertenece a (un lenguaje²⁷).

(Introducidos²⁸) los conceptos de complejidad, en la próxima sección se presenta el problema de la satisfacibilidad booleana, cuyo estudio mediante formalismos de la teoría de lenguajes motiva este trabajo.

1.4. Problema de la satisfacibilidad booleana

El problema de la satisfacibilidad booleana (SAT) es un problema de la teoría de la computación y la lógica matemática [6]. Consiste en determinar si existe una asignación de valores a las variables de (una fórmula booleana^{Q33}) tal que la expresión sea verdadera.

En esta sección se presentan los elementos del SAT y se introducen las variantes polinomiales que se utilizan en los capítulos posteriores. La sección se organiza en dos partes. Primero se presentan los elementos básicos del SAT. Después se analiza el estatus del SAT como problema NP -Completo junto con sus variantes polinomiales.

1.4.1. Elementos del SAT

A continuación se presentan los elementos del SAT: variables booleanas, literales, cláusulas, fórmulas en forma normal conjuntiva y fórmulas booleanas equivalentes.

Q28: ¿Qué es esto?

Q29: ¿Qué es esto?

Q31: ¿Qué es esto?

23: Mmmm... creo que con ese nombre no lo vamos a usar mucho, así que vamos a quedarnos solo con problema de la palabra, a no ser que sea vital para el resto del trabajo

24: ¿qué símbolos?

25: ¿cuál?

Q32: ¿Qué es esto?

26: ¿asociada a qué?

27: ¿qué lenguaje?

28: Voz pasiva

Q33: ¿Qué es esto?

- **Variables booleanas:** una variable booleana es una variable que puede tomar uno de dos valores: *true* o *false*. [Aquí propondría usar verdadero y falso en español. Realmente no hay un motivo de peso para usarlas en inglés... No es solo aquí, sino en todo el documento :-).]
- **Literales:** un literal es una variable booleana o su negación. Formalmente, si x es una variable booleana, entonces x y $\neg x$ son literales. Un literal toma los valores *true* o *false* según la asignación de valores a las variables.
- **Cláusulas:** una cláusula es una disyunción, con el operador (**OR**²⁹), de uno o más literales. Por ejemplo, la cláusula $(x \vee \neg y \vee z)$ es una disyunción de tres literales: x , $\neg y$ y z . Una cláusula es verdadera si al menos uno de sus literales es verdadero; es falsa si todos sus literales son falsos.
- **Fórmulas en forma normal conjuntiva:** una fórmula booleana en forma normal conjuntiva, abreviada CNF, es una conjunción con el operador (**AND**³⁰) de cláusulas. Por ejemplo,

$$(x \vee \neg y \vee z) \wedge (\neg x \vee y) \wedge (x \vee \neg z)$$
 es una fórmula en CNF con tres cláusulas y tres variables.
- **Fórmulas booleanas equivalentes:** dos fórmulas booleanas son equivalentes si, para cualquier asignación de valores a sus variables, ambas producen el mismo resultado lógico. Por ejemplo, las fórmulas $x \vee (y \wedge z)$ y $(x \vee y) \wedge (x \vee z)$ son equivalentes. [¿y esto por qué es relevante?]

29: A esto dale alguna tipografía relevante, además de las mayúsculas.

30: tipografía adecuada

Una **interpretación** de una fórmula booleana es una asignación de valores *true* o *false* a cada una de sus variables. (Para cualquier fórmula booleana existe una fórmula booleana equivalente en CNF, y el algoritmo para encontrarla es polinomial [6]³¹). Por esta razón, en el resto del trabajo se asume sin pérdida de generalidad que toda fórmula booleana está en CNF.

Definición 1.10. El problema de la **satisfacibilidad booleana**, o SAT, consiste en determinar si existe una asignación de valores *true* o *false* a las variables de una fórmula booleana, tal que la fórmula completa sea verdadera.

Una fórmula booleana en CNF es **satisfacible** si existe una asignación de valores a las variables tal que todas las cláusulas de la fórmula sean verdaderas simultáneamente.

Una vez definido el problema y sus elementos, a continuación se sitúa al SAT dentro de la clasificación de complejidad y se describen sus variantes polinomiales.

1.4.2. SAT como problema NP-Completo y variantes polinomiales

El SAT es el primer problema demostrado como NP-Completo [6] y juega un rol central en la teoría de la complejidad computacional. El SAT pertenece a la clase NP porque, dada una asignación de valores a las variables de la fórmula, se puede verificar en tiempo polinomial si la asignación satisface la fórmula. La demostración de que el SAT es NP-Completo fue una de las contribuciones principales de Stephen Cook en 1971 [2], y marcó el inicio de la teoría de la NP-Complejidad.

31: Aquí Jossué me rectificó que eso no es exactamente así, lo que me dijo que es que una es satisfacible si y solo si la otra lo es, pero no es que sean exactamente equivalentes para todos los valores de variables. ¿Puedes ponerte de acuerdo con él? O sea, sigue siendo válido que se puede asumir que todo está en CNF, pero no son exactamente equivalentes, dice Jossué que está dicho en algún lugar.

Un SAT en el que cada cláusula tiene exactamente n literales distintos se denomina n -SAT. Para el problema 2-SAT existe una solución polinomial que determina si la fórmula booleana es satisfacible o no [5]. Para el problema 3-SAT no se conoce ningún algoritmo polinomial que permita determinar si una fórmula booleana es satisfacible o no [6]. Toda fórmula booleana del problema n -SAT se puede reducir a una fórmula equivalente del problema 3-SAT, por lo que SAT y 3-SAT son equivalentes en términos de complejidad computacional [6].

Aunque no se conoce algoritmo polinomial para resolver el SAT general, existen variantes particulares que sí admiten solución polinomial como (- En este trabajo se consideran tres:) 2-SAT, Horn-SAT y XOR-SAT.

El problema **2-SAT** es una variante del SAT donde cada cláusula contiene exactamente dos literales. El problema 2-SAT admite solución en tiempo polinomial mediante una modelación basada en grafos, con algoritmos como la detección de componentes fuertemente conexas en el grafo de implicación [5]. El 2-SAT es la variante polinomial que se utiliza como referencia en los capítulos posteriores para analizar subclases polinomiales del SAT descritas mediante formalismos de la teoría de lenguajes.

El problema **Horn-SAT** es una variante del SAT donde cada cláusula tiene a lo sumo un literal positivo. El problema Horn-SAT admite solución en tiempo polinomial mediante el algoritmo de resolución de Horn [3]. **NUEVO PÁRRAFO.**

El problema **XOR-SAT** es una variante del SAT donde cada cláusula representa una operación XOR sobre sus literales. El problema XOR-SAT admite solución en tiempo polinomial al transformarlo en un sistema de ecuaciones lineales sobre $(GF(2))^{Q^{34}}$ y aplicar eliminación de Gauss [1].

Q34: ¿Qué es esto?

AQUÍ FALTA UNA TRANSICIÓN DE LOS CAOS PARTICULARES DE SAT A LAS CONCLUSIONES DEL CAPÍTULO.

En el próximo capítulo se presentan las gramáticas de concatenación de rango, se analizan sus propiedades y limitaciones, y se revisan los antecedentes directos que abordan el SAT mediante formalismos de la teoría de lenguajes, con énfasis en los trabajos previos realizados en la Facultad de Matemática y Computación de la Universidad de La Habana.

Bibliografía

- [1] Armin Biere, Marijn Heule, Hans van Maaren, and Toby Walsh, editors. *Handbook of Satisfiability*, volume 185 of *Frontiers in Artificial Intelligence and Applications*. IOS Press, Amsterdam, 2009.
- [2] Stephen A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing (STOC '71)*, pages 151–158, Shaker Heights, Ohio, USA, 1971. ACM.
- [3] William F. Dowling and Jean H. Gallier. Linear-time algorithms for testing the satisfiability of propositional Horn formulae. *Journal of Logic Programming*, 1(3):267–284, 1984.
- [4] A. Fernández Arias. El problema de la satisfacibilidad booleana libre del contexto. Trabajo de diploma, Facultad de Matemática y Computación, Universidad de La Habana, 2007.
- [5] Steven Halim, Felix Halim, and Suhendry Effendy. *Competitive Programming 4 – Book 2: The Lower Bound of Programming Contests in the 2020s*. Lulu.com, 4 edition, 2020.
- [6] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Pearson Addison-Wesley, Boston, MA, 3 edition, 2006.
- [7] Tim Hunter. The Chomsky hierarchy. In Nicholas Allott, Terje Lohndal, and Georges Rey, editors, *A Companion to Chomsky*, chapter 5, pages 74–95. Wiley-Blackwell, 2021.